

Lab 8 EE 446

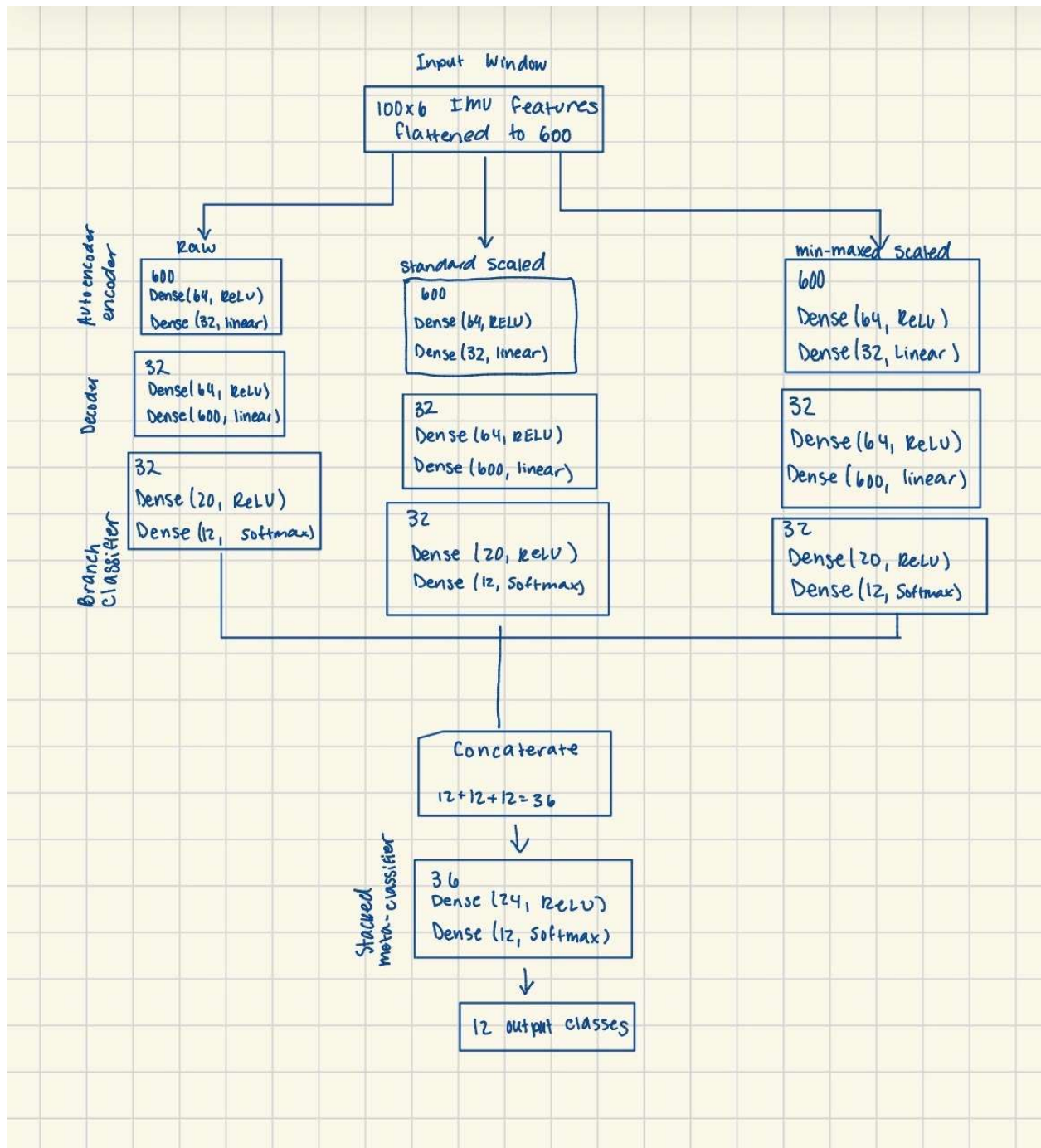
Part 1: Edge Impulse

Link: <https://studio.edgeimpulse.com/public/1013671/live>

```
Sampling...
Timing: DSP 159 ms, inference 553 us, anomaly 0 ms, postprocessing 66 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 68 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 160 ms, inference 574 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
```

Part 2: Discussion Questions

Question 1: Model architecture and ensemble flow



Question 2: Pruning and QAT methodology

- How the final model is converted to an int8 TFLite model and why representative data is needed.
- How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.

Pruning is applied using the TensorFlow Model Optimization Toolkit (TF-MOT) function `prune_low_magnitude()`. Pruning works by forcing parameters to go to zero to remove the weights that are less important. The pruning uses a polynomial sparsity schedule, so sparsity grows throughout the training process, from 10% to 80% over 500 pruning steps. This lets models slowly adjust to the sparsity instead of having a very sudden change.

The wrappers are removed so that the model size can decrease. With the wrappers, the model size is not decreased substantially, so the wrappers get removed before QAT. Quantization libraries also don't expect wrappers since they expect unmodified Keras models.

QAT is applied with `quantize_model()`. It inserts quantization simulations so that the noise introduced by quantization can be used during the training. When it is quantized, the quantified weights are trained to maximize accuracy with the noise.

Additional mask enforcement happens since the model is pruned. Without it, QAT would not need it. Since pruning makes many parameters go to zero, which makes the model smaller, the QAT while training can grow the parameters again. By applying the masks after QAT, the pruned parameters are forced to stay at zero.

The final model is converted to an int8 TFLite model using a TFLite converter. A representative data generator is used to calibrate the quantization process — it samples the range of real input data so that the converter can determine appropriate scaling factors, allowing the model to be accurately quantized from float32 to int8.

When this is deploying to a resource constrained device, the amount of memory that can be used for holding the model and resources for inference are much smaller than the computer the model was built and trained on. Using int8 and pruning quantization you are able to get high accuracy with small memory by building and training a good model and using these models to find the best parameters to make them smaller. The original accuracy can also be preserved.

Question 3: Arduino deployment and real-world behavior

```

Final prediction: Lying down
Class index: 2
Confidence score: 0.5078
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.3281, 0.3281, 0.0000, 0.0000, 0.0000, 0.3281, 0.0000, 0.0000, 0.0000, 0.0117, 0.007
Standard-scaled model scores: 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0
Min-max-scaled model scores: 0.0000, 0.0000, 0.4180, 0.0000, 0.0977, 0.0898, 0.2031, 0.1562, 0.0000, 0.0000, 0.
Meta-classifier scores: 0.3437, 0.0234, 0.5078, 0.0117, 0.0000, 0.0156, 0.0000, 0.0078, 0.0000, 0.0117, 0.0703,
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.5078
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.3281, 0.3281, 0.0000, 0.0000, 0.0000, 0.3281, 0.0000, 0.0000, 0.0000, 0.0117, 0.007
Standard-scaled model scores: 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0
Min-max-scaled model scores: 0.0000, 0.0000, 0.4180, 0.0000, 0.0977, 0.0898, 0.2031, 0.1562, 0.0000, 0.0000, 0.
Meta-classifier scores: 0.3437, 0.0234, 0.5078, 0.0117, 0.0000, 0.0156, 0.0000, 0.0078, 0.0000, 0.0117, 0.0703,
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.5078
-----

```

The resting board output is acceptable — the board is completely still and correctly predicted as lying down. When I began moving the board, however, only two classes were ever predicted: "Standing up" and "Knees bending." No other classes appeared.

"Standing up" was triggered by any horizontal movement, while "Knees bending" corresponded more to vertical movement. The model classifies standing up different than I would, since it seems to classify it as the physical motion of standing up, when I expected it to be when the board was stationary.

Clearer usage instructions would likely improve deployment reliability — particularly guidance on how the board should be oriented and moved in practice. This shows how important understanding your dataset is and is just as critical as building the model itself.

Question 4: Deployment Behavior on the Arduino

Deployment revealed several unexpected predictions. The board predominantly predicted "Standing up," but only while in constant motion, and just 3 of the 12 classes were predicted at all. This is likely due to a fundamental mismatch between how the board was moved during testing and how the original training data was collected. The training data

was most likely recorded from a sensor worn by a person, whereas the board was being moved freely on a cord — a very different motion profile.

Additional contributing factors may include differences in board orientation and sensor noise between the deployed sensor and the original. These discrepancies could cause the model to favor certain classes over others, and it's possible that the movements performed during testing simply failed to activate the feature patterns associated with the remaining classes.